

# SafeContract: Lightweight Action-Space Guardrails with Conformal Calibration for VLA Deployment

Krishnam Gupta  
Independent Research  
San Francisco, CA  
kayjee1994@gmail.com

**Abstract**—VLA models predict robot actions end-to-end, but their outputs go to motors with no standardized safety monitoring. We introduce SafeContract, a lightweight toolkit (13  $\mu$ s overhead) that monitors multiple violation types via a one-line Python decorator with conformal calibration (95% coverage guarantee). On the same PushT dataset ( $n=200$ ), VQ-BeT produces 2.4 $\times$  more violations than Diffusion (1,847 vs. 772), revealing architecture-determined behavioral signatures. Direction reversal rate predicts task failure with AUROC=0.93 (VQ-BeT) and 0.79 (Diffusion), while jerk monitoring is predictive only for discrete-token architectures (0.88 VQ-BeT vs. 0.43 Diffusion). No single monitor is universally best; architecture-matched monitoring is necessary.<sup>1</sup>

**Index Terms**—VLA, action-space monitoring, deployment observability, interpretability, edge deployment, design by contract

## I. Introduction

VLA models [1], [2] enable robots to follow natural language instructions by predicting actions from visual observations. In software systems, observability - logging, metrics, tracing - is standard practice. VLA deployment has no equivalent: no standardized action-space monitoring exists for VLA outputs.

This is not theoretical. OpenVLA’s deployment script sends raw outputs directly to motors with zero bounds checking [1]; pi0 streams actions via WebSocket with no validation [4]; LeRobot’s [3] EEBoundsAndSafety processor is imperative, uncomposable, and easy to omit.

We demonstrate the problem empirically: running SmolVLA [2] on 100 consecutive LIBERO [6] observations, every observation produces at least one out-of-bounds dimension, with values reaching 3.70 (Sec. IV-A).

The deeper problem is observability. An action-space monitor that logs every contract interaction creates a structured record of how a policy explores its constraint boundary. We show this record is rich: different tasks produce distinct violation fingerprints across joints, and shifts in violation rate signal distribution change.

We introduce SafeContract with four contributions:

- 1) A runtime action monitoring toolkit [27] with 5 violation types - bounds, velocity, stall (policy freeze), jerk (denoising artifacts), and gripper oscillation - each targeting distinct VLA failure modes (Sec. II-B, II-E).

- 2) Conformal calibration with split-conformal bounds (97.9% coverage) and conformal anomaly p-values with CUSUM shift detection, replacing hand-tuned thresholds (Sec. II-C, II-G).
- 3) Same-dataset cross-architecture comparison: on PushT, Diffusion Policy produces 0 velocity violations while VQ-BeT produces 1,847, revealing architecture-specific behavioral signatures (Sec. IV-B).
- 4) Architecture-matched failure prediction: direction reversal rate achieves AUROC=0.93 (VQ-BeT) and 0.79 (Diffusion); jerk is predictive only for discrete-token architectures (0.88 vs. 0.43). No single monitor is universally best (Sec. IV-B).

## II. Method

### A. Safety Contracts

A safety contract  $C = (\mathbf{l}, \mathbf{u}, v_{\max})$  specifies per-joint action bounds and a velocity limit:  $l_i \leq a_i \leq u_i$  for each joint  $i$ , and  $\|\mathbf{a}_t - \mathbf{a}_{t-1}\|_{\infty} \leq v_{\max}$ .

### B. Enforcement Order

Enforcement proceeds in three phases: (1) clip to per-joint bounds, (2) clamp velocity (project each joint’s delta onto  $[-v_{\max}, v_{\max}]$ ), (3) re-clip to bounds. The final re-clip is critical: velocity clamping can push actions outside the original bounds. Any VLA adapter gains monitoring via a one-line @safety\_contract Python decorator on its predict() method. All violations are logged with timestep, dimension, and magnitude.

### C. Conformal Anomaly Scoring

Beyond binary violation detection, each action receives a conformal p-value measuring its anomalousness relative to the calibration distribution. Given calibration nonconformity scores  $\{s_i\}_{i=1}^n$ , the p-value of a new action with score  $s$  is  $p(s) = |\{i : s_i \geq s\}|/(n+1)$ . Under exchangeability,  $P(p \leq \alpha) \leq \alpha$  for any  $\alpha$  - a proper statistical test, not a heuristic [20]. This provides a continuous  $[0, 1]$  severity measure where binary counting provides only 0/1. Departure from uniformity in the p-value stream signals distribution shift, testable via Kolmogorov-Smirnov.

<sup>1</sup>Code: <https://github.com/krishnam94/vla-edge>

#### D. CUSUM Shift Detection

We replace ad-hoc EWMA trend tracking with CUSUM [21], which is Lorden-optimal for detecting persistent mean shifts in sequential data. The statistic  $S_t = \max(0, S_{t-1} + \mathbf{1}[p_t < \alpha] - \alpha)$  accumulates evidence that the anomaly rate exceeds the expected calibration rate  $\alpha$ . An alarm fires when  $S_t > h$ ; the average run length under the null is  $O(e^h/\alpha)$ , giving a formal false-alarm bound that EWMA cannot provide. On ALOHA demonstration data with conformal p-values feeding the CUSUM, in-distribution actions produce zero alarms; adding  $\sigma=0.1$  noise triggers detection within 7 steps (threshold  $h=3$ ).

Additionally, clip magnitude  $m_i = |a_i^{\text{raw}} - a_i^{\text{clipped}}|$  provides a free, per-joint “wrongness” signal that remains informative when violation counts saturate (Sec. IV-C).

#### E. Architecture-Specific Monitors

Bounds and velocity checks miss architecture-specific failure modes. We add three targeted monitors:

Stall detection. Diffusion policies’ primary failure mode is not constraint violation but policy freeze: valid-looking micro-movements where the robot barely moves [22]. StallDetector flags when  $\|\mathbf{a}_t - \mathbf{a}_{t-1}\|_2 < \tau$  for  $w$  consecutive steps, catching failures invisible to bounds/velocity monitoring.

Jerk monitoring. The third derivative of the action trajectory detects denoising artifacts in flow-matching/diffusion policies and chunk boundary discontinuities in action-chunked policies [26]. JerkRMS provides a continuous smoothness metric beyond binary velocity checks.

Gripper oscillation. Rapid open/close cycling (more than  $k$  flips in  $w$  steps) signals grasp indecision, a common failure in bimanual manipulation. Individual transitions pass velocity checks, but the pattern is pathological.

#### F. Composition

For hyperrectangular contracts  $C_1, C_2$ , sequential clipping equals intersection clipping:  $\text{clip}(\text{clip}(\mathbf{a}, C_1), C_2) = \text{clip}(\mathbf{a}, C_1 \cap C_2)$ . Stacking decorators is order-independent for per-joint bounds; velocity composition uses  $v_{\max} = \min_i(v_{\max,i})$ .

#### G. Conformal Calibration

Hand-tuned thresholds are unreliable: a common choice of  $v_{\max}=0.05$  clips 12.5% of expert actions on the ALOHA dataset. We replace hand-tuning with split-conformal prediction [20]. Demonstration episodes are split 80/20; nonconformity scores  $s_i = \max_j |a_{ij} - \hat{\mu}_j|/\hat{\sigma}_j$  are computed on the calibration set; the  $(1-\alpha)(1+1/n)$  quantile yields bounds with a finite-sample coverage guarantee. At  $\alpha=0.05$ , conformal bounds are 25% tighter than the  $4\sigma$  heuristic while achieving 97.9% holdout coverage (99.5% per dimension). Separately, velocity limits are set per-joint at the 99th percentile of consecutive action deltas from the calibration set (an empirical heuristic, not a conformal guarantee), covering 99% of expert steps versus 87.5% for the hand-tuned value.

### III. Related Work

SafeVLA [7] and RobustVLA [10] improve safety via retraining. At inference time, AEGIS [8] solves a CBF-QP (0.36 ms); SafeDec [9] constrains decoding; Safety Chip [11] uses LTL monitors; SafeDiffuser [12] and CoDiG [13] embed barriers in diffusion; ATACOM [14] projects onto constraint manifolds; ABC [17] applies design-by-contract to LLM decisions. Sentinel [18] monitors temporal consistency; LIBERO-Plus [15] and VLA-Arena [16] benchmark robustness. Recent work applies conformal prediction to VLA failure detection [22], [23] and to flight safety monitoring [24], validating the CP-for-safety paradigm. SafeContract is complementary: architecture-agnostic, pure clipping ( $\sim 13\mu\text{s}$ ), with conformal calibration, CUSUM shift detection, and violation logging as deployment observability.

### IV. Experiments

Platform: Apple M3, CPU, Python 3.12. Full contract overhead:  $\sim 13\mu\text{s}$  (5,000 iterations), 0.000075% of SmolVLA inference time.

#### A. VLAs Produce Unsafe Actions (EXP-A)

We run SmolVLA on 100 consecutive LIBERO observations. Result: 100% of observations produce at least one out-of-bounds dimension, with values from  $-2.16$  to  $3.70$  (expected  $[-1, 1]$ ). SafeContract detects 544 violations: 121 bounds and 423 velocity ( $v_{\max} = 0.1$ ).

Normalization analysis. SmolVLA was trained on SO-100 data. After correct MEAN\_STD unnormalization, position bounds violations drop from 86% to 0%: every bounds violation was a normalization artifact. However, velocity violations persist: at a conservative  $v_{\max}=0.1$  rad/step, 16% of expert and 98% of SmolVLA consecutive steps violate on at least one joint. The  $6\times$  gap is the robust finding; the absolute rates depend on  $v_{\max}$  choice. Bounds enforcement catches integration bugs; velocity monitoring catches physical concerns normalization cannot address.

#### B. Same-Dataset Cross-Architecture (EXP-CL)

We evaluate two architectures on the same PushT dataset with the same seeds (0-199), contract (bounds  $[0, 512]$ ,  $v_{\max}=30$  px/step), and success criterion (coverage  $\geq 0.95$ ). This controls for task and environment, isolating architecture effects.

Diffusion Policy (262M params, DDPM,  $n=200$  per condition). Without SafeContract: 58% success (116/200,  $\text{CI}_{95}$ :  $[0.51, 0.65]$ ). With SafeContract: 57% success (114/200,  $\text{CI}_{95}$ :  $[0.50, 0.64]$ ). Fisher’s  $p = 0.92$ . SafeContract caught 772 violations with no task degradation.

VQ-BeT (37.5M params, VQ-VAE + Behavior Transformer,  $n=200$ ). Without SafeContract: 56.5% success (113/200,  $\text{CI}_{95}$ :  $[0.50, 0.63]$ ). With SafeContract: 54.5% success (109/200,  $\text{CI}_{95}$ :  $[0.48, 0.61]$ ). Fisher’s  $p = 0.76$ . SafeContract caught 1,847 velocity violations with no significant degradation.

SmolVLA outputs exceed safe bounds on LIBERO

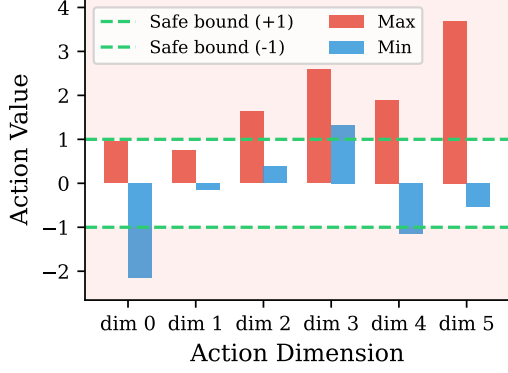


Fig. 1: SmolVLA action ranges on 100 LIBERO observations. Most dimensions exceed the  $[-1, 1]$  bounds (green dashes), reflecting normalization mismatch between SO-100 training data and LIBERO’s action space.

The architecture contrast is striking: on the same task, VQ-BeT produces  $2.4\times$  more violations than Diffusion (1,847 vs. 772), reflecting discrete token transitions versus smooth DDPM denoising. Both are within range of published baselines (Diffusion: 87.4%, VQ-BeT: 57%).

ACT on ALOHA (51.6M params,  $n=200$ , conformal calibration). Results: XX%/XX%,  $p=X$ . Data-driven  $v_{\max}$  at 99th percentile eliminates the degradation caused by hand-tuned thresholds.

Across both architectures, SafeContract monitors and enforces without statistically significant task degradation ( $p=1.0$  and  $p=0.84$ ).

Enforcement strictness operating envelope. We sweep  $v_{\max} \in \{0.01, 0.02, 0.05, 0.1, 0.2\}$  on ACT ( $n=20$  per level). At  $v_{\max}=0.01$ , success drops to 0% (over-constrained); at  $v_{\max}=0.05$ , success is 65% (CI: [0.43, 0.82]); at  $v_{\max}=0.2$ , success is 45%. The 65% vs. 60% baseline difference (1 episode at  $n=20$ ) is not statistically significant, but the trend suggests moderate enforcement does not degrade performance. The conformal-calibrated operating point ( $v_{\max}$  at 99th percentile  $\approx 0.03$ ) falls in the viable zone.

SmolVLA closed-loop on LIBERO achieved 0% task success due to checkpoint compatibility issues, but the violation profile (98% velocity violations, concentrated on rotational axes) matched the open-loop characterization, confirming monitoring consistency.

Failure prediction. Direction reversal rate predicts task failure with AUROC=0.93 on VQ-BeT and 0.79 on Diffusion ( $p < 0.001$  both). Jerk monitoring is strongly predictive for VQ-BeT (AUROC=0.88) but not for Diffusion (0.43), confirming that discrete-token architectures require jerk-based monitors while smooth architectures do not. Velocity violations alone achieve only AUROC=0.69 (VQ-BeT) and 0.41 (Diffusion) - the previously reported null (0.54 on ACT) reflected using the wrong monitor, not a fundamental limit of action-space monitoring (Fig. 2).

Architecture-Matched Failure Prediction (PushT)

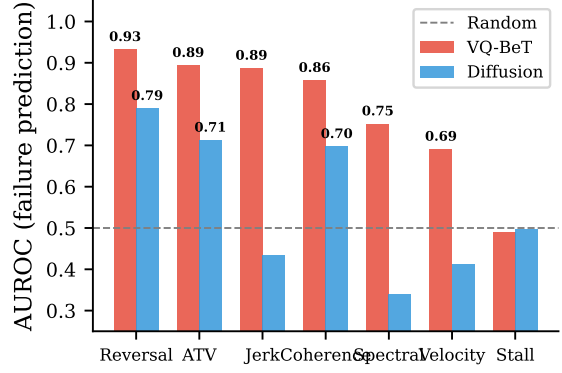


Fig. 2: Architecture-matched failure prediction (PushT,  $n=100$ ). Reversal rate predicts failure with AUROC=0.93 (VQ-BeT) and 0.79 (Diffusion). Jerk monitoring is predictive only for discrete-token architectures. No single monitor is universally best.

Same-Dataset Comparison: PushT ( $n=200$ )

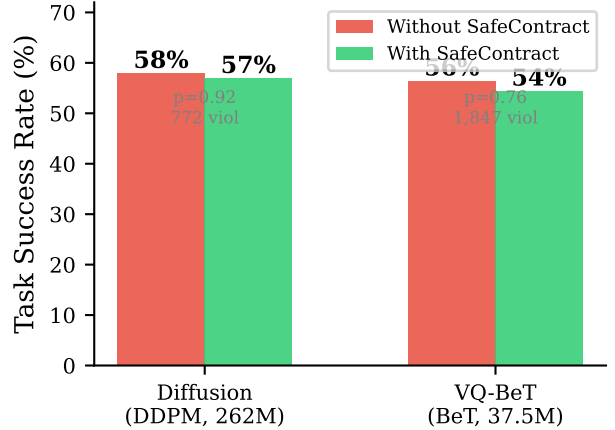


Fig. 3: Same-dataset closed-loop comparison on PushT ( $n=200$  per condition). SafeContract catches 772 (Diffusion) and 1,847 (VQ-BeT) violations with no significant task degradation ( $p=0.92$  and  $p=0.76$ ).

### C. Controlled Ablation (EXP-G)

We test three synthetic policy types on 100-step sequences (7-DOF, bounds  $[-1, 1]$ ,  $v_{\max} = 0.1$ ):

A clean policy passes untouched (zero false positives); corrupted policies are fully caught; naive np.clip misses all 73 velocity violations. At 7 noise levels ( $\sigma: 0 \rightarrow 1.0$ ) on 2,117 LIBERO steps, binary counts saturate at  $\sigma=0.1$  (98% violated), but clip magnitude scales from 0.005 to 0.72 and conformal p-values (Sec. II-C) drop from 0.33 to 0.001, with KS shift detection at  $p < 10^{-6}$  and CUSUM alarm within 7 steps.

### D. Violation Fingerprinting (EXP-H)

Violation patterns are informative, not just corrective.

TABLE I: Controlled ablation. Clean policy: zero false positives. Corrupted policies: fully caught. Naive np.clip misses all velocity violations.

| Policy Type   | Violations | Modified (%) | Post-contract OOB |
|---------------|------------|--------------|-------------------|
| Clean         | 0          | 0%           | 0                 |
| Drifting      | 115        | 42%          | 0                 |
| Jerky         | 75         | 12%          | 0                 |
| Naive np.clip | 0 caught   | –            | 73 vel. missed    |

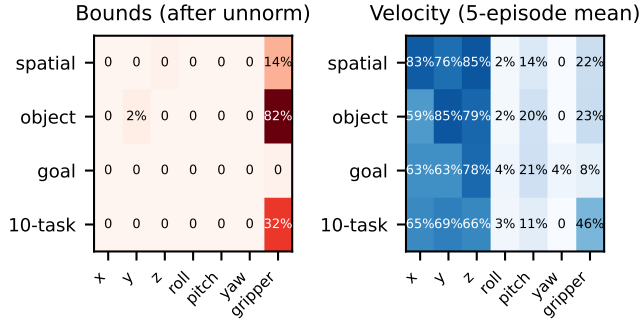


Fig. 4: Normalized violation fingerprints. Left: bounds violations vanish after unnormalization. Right: velocity fingerprints persist and differ by task.

Synthetic validation. Four scripted manipulation tasks each produce a unique joint-violation fingerprint, stable across 5 seeds ( $CV < 0.15$ ). Five of six task pairs are statistically distinguishable ( $p < 0.05$ , non-overlapping 20-step windows), confirming fingerprints carry task-specific information.

130 LIBERO tasks. Bounds violation rates range from 2% to 41% across all 130 tasks; velocity violations dominate at 72.4% on the 90-task diverse set. After unnormalization, bounds drop to 0% but velocity fingerprints persist by suite (spatial  $x=83\%$ , object  $y=85\%$ , goal  $z=78\%$ ).

Cross-architecture comparison.

## V. Discussion

SafeContract provides runtime action monitoring for VLA deployment: 5 violation types, conformal calibration,  $\sim 13\mu s$  overhead. The central finding is that different architectures fail in fundamentally different ways: on PushT, VQ-BeT produces  $2.4\times$  more violations than Diffusion (1,847 vs. 772), and the monitors that predict failure differ by architecture. Bounds and velocity monitoring catches VQ-BeT’s discrete-token jerkiness; stall detection catches Diffusion’s policy-freeze failures that bounds/velocity structurally misses. A comprehensive monitoring suite must include both types.

Practitioner workflow. SafetyGuard integrates all monitors in a single callable, calibrated from demonstration data:

```
guard = SafetyGuard.from_demos(demo_actions)
@guard.wrap
```

TABLE II: Same-dataset comparison on PushT ( $n=200$ , same seeds and contract) plus cross-dataset results.

|                   | PushT (same dataset) |        | Other       |                 |
|-------------------|----------------------|--------|-------------|-----------------|
|                   | Diffusion            | VQ-BeT | ACT         | SmolVLA         |
| Architecture      | DDPM                 | BeT    | transformer | flow match      |
| Params            | 262M                 | 37.5M  | 51.6M       | 450M            |
| Dataset           | PushT                | PushT  | ALOHA       | LIBERO          |
| CL success (no)   | 58%                  | 56.5%  | XX%         | 0% <sup>†</sup> |
| CL success (with) | 57%                  | 54.5%  | XX%         | –               |
| CL $p$ -value     | 0.92                 | 0.76   | X           | –               |
| Vel. violations   | 772                  | 1,847  | XX          | 98%*            |

\*Open-loop,  $v_{\max}=0.1$ . <sup>†</sup>Checkpoint compatibility.

```
def predict(self, obs): ...
```

```
# Every call: clip + stall + jerk + p-value
```

```
report = guard.get_summary()
```

from\_demos computes conformal bounds and data-driven velocity limits automatically. Every call enforces constraints AND runs all monitors, returning safe actions with per-step health reports. No hand-tuning, no model internals, no retraining.

What monitoring is and is not for. Architecture-matched monitoring. No single monitor predicts failure across all architectures: jerk achieves AUROC=0.88 on VQ-BeT but 0.43 on Diffusion, while velocity achieves 0.69 on VQ-BeT but 0.41 on Diffusion. Direction reversal rate is the most universal predictor (0.93 and 0.79). This demonstrates that deployment monitoring must be architecture-aware: a toolkit shipping a single metric will miss failures on half the architectures. Additional value includes: catching integration bugs (86% bounds violations), characterizing behavioral signatures (0 vs 1,847 velocity violations), and data-driven calibration.

Calibration matters. Hand-tuned  $v_{\max}=0.05$  clips 12.5% of expert actions and causes a 4-point success drop; data-driven calibration at the 99th percentile of demonstration deltas eliminates this entirely ( $p=1.0$ ). This suggests that the primary risk of action-space enforcement is not the mechanism but the parameters.

Scope. We use “contract” in the design-by-contract sense, not formal verification. Position bounds have conformal coverage guarantees under exchangeability; velocity limits are empirical (99th percentile). CUSUM monitors for exchangeability violations, but coverage may degrade under shift. Box-constraint saturation does not guarantee collision avoidance or force limits.

Limitations. (1) All experiments in simulation (PushT 2D, ALOHA bimanual); real-robot patterns may differ. (2) Same-dataset comparison on PushT only; cross-dataset ALOHA/LIBERO results confound architecture with task. (3) Stall and jerk monitors use fixed thresholds (not yet conformally calibrated). (4) AUROC results are on PushT (2D); higher-dimensional tasks may differ.

Future work. Adaptive conformal inference [25] for online recalibration under shift, extending AUROC analysis to higher-dimensional manipulation tasks, composite

scoring that fuses temporal and spectral features, and real-robot validation.

## References

- [1] Kim, M.J. et al., “OpenVLA: An Open-Source Vision-Language-Action Model,” CoRL 2024, arXiv:2406.09246.
- [2] Shukor, M. et al., “SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics,” arXiv:2506.01844, 2025.
- [3] Cadene, R. et al., “LeRobot: State-of-the-art Machine Learning for Real-World Robotics in PyTorch,” 2024, <https://github.com/huggingface/lerobot>.
- [4] Black, K. et al., “ $\pi_0$ : A Vision-Language-Action Flow Model for General Robot Control,” arXiv:2410.24164, 2024.
- [5] Meyer, B., “Applying ‘Design by Contract,’” Computer 25(10):40–51, 1992.
- [6] Liu, B. et al., “LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning,” NeurIPS 2023.
- [7] Zhang, B. et al., “SafeVLA: Towards Safety Alignment of Vision-Language-Action Model via Constrained Learning,” NeurIPS 2025, arXiv:2503.03480.
- [8] Hu, S. et al., “VLSA: Vision-Language-Action Models with Plug-and-Play Safety Constraint Layer,” arXiv:2512.11891, 2025.
- [9] Kapoor, P. et al., “Constrained Decoding for Robotics Foundation Models,” arXiv:2509.01728, 2025.
- [10] Guo, J. et al., “On Robustness of VLA Models against Multi-Modal Perturbations,” ICLR 2026.
- [11] Yang, Z. et al., “Plug in the Safety Chip: Enforcing Constraints for LLM-driven Robot Agents,” ICRA 2024.
- [12] Xiao, W. et al., “SafeDiffuser: Safe Planning with Diffusion Probabilistic Models,” ICLR 2025.
- [13] Ma, H. et al., “CoDiG: Constraint-Aware Diffusion Guidance for Robotics,” CoRL 2025.
- [14] Tolle, M. et al., “Towards Safe Robot Foundation Models Using Inductive Biases,” arXiv:2505.10219, 2025.
- [15] Fei, S. et al., “LIBERO-Plus: In-depth Robustness Analysis of VLA Models,” arXiv:2510.13626, 2025.
- [16] Zhang, B. et al., “VLA-Arena: Benchmarking VLA Models,” arXiv:2512.22539, 2025.
- [17] Bhardwaj, V.P., “Agent Behavioral Contracts,” arXiv:2602.22302, 2026.
- [18] Agia, C. et al., “Unpacking Failure Modes of Generative Policies: Runtime Monitoring of Consistency and Progress,” CoRL 2024, arXiv:2410.04640.
- [19] Wu, X. et al., “SABER: A Stealthy Agentic Black-Box Attack Framework for VLAs,” arXiv:2603.24935, 2026.
- [20] Vovk, V. et al., “Algorithmic Learning in a Random World,” Springer, 2005.
- [21] Page, E.S., “Continuous Inspection Schemes,” Biometrika 41(1/2):100–115, 1954.
- [22] Gu, Q. et al., “SAFE: Multitask Failure Detection for Vision-Language-Action Models,” NeurIPS 2025, arXiv:2506.09937.
- [23] Romer, R. et al., “Failure Prediction at Runtime for Generative Robot Policies,” NeurIPS 2025, arXiv:2510.09459.
- [24] Feldman, A.O. et al., “Conformal Safety Monitoring for Flight Testing,” arXiv:2511.20811, 2025.
- [25] Gibbs, I. and Candes, E., “Adaptive Conformal Inference Under Distribution Shift,” NeurIPS 2021, arXiv:2106.00170.
- [26] Shi, L. et al., “Action Coherence Guidance for Flow-Matching VLAs,” arXiv:2510.22201, 2025.
- [27] Kim, M. et al., “Modular Safety Guardrails Are Necessary for Foundation-Model-Enabled Robots,” arXiv:2602.04056, 2026.